

GitLab Release Automation Tool – System Design Document

1. Overview

This document describes the architecture and design of a GitLab Release Automation Tool used to manage software releases across multiple repositories.

The tool automates identifying release candidates, creating release branches, merging changes, and creating merge requests using GitLab APIs.

The system is designed for approximately 25–30 repositories under a single product in a GitLab group.

2. Goals

- Automate release candidate identification
- Automate creation and synchronization of release branches
- Automate merge request creation to master
- Provide a UI for monitoring release progress
- Maintain release history
- Allow retries for failed operations
- Handle merge conflicts gracefully

3. NonGoals

- Replace GitLab CI/CD
- Replace GitLab's native merge process
- Manage production deployments

4. System Architecture

The system is implemented as a single application with two major components:

Frontend:

Angular + Angular Material

Provides UI for managing releases and viewing stage results.

Backend:

Python FastAPI service responsible for GitLab API communication and release workflow orchestration.

Storage:

JSON files used for storing repository reference data and release state.

5. Repository Structure

GitLab Structure:

Group: truenta

Product: pioneer

Repositories: 25–30

Branches:

master

develop

Developers merge code into develop branch. The release manager creates release branches and merge requests.

6. Release Workflow Stages

Stage 1 – Identify Release Candidates

User provides repository names. The system validates them against a reference JSON file and marks valid repositories as release candidates.

Stage 2 – Release Branch Management

For each repository:

- Create release/ branch if it does not exist
- Merge develop into release branch if updates exist
- Detect merge conflicts

Stage 3 – Merge Request Creation

Create merge requests from:

release/ → master

if an MR does not already exist.

7. Data Storage Strategy

Reference JSON:

Stores metadata about all repositories.

Example:

name

project_id

path_with_namespace

web_url

default_branch

develop_branch

Release JSON:

Each release has a separate JSON file storing stage results, repository state, and timestamps.

8. Backend API Endpoints

GET /api/repos/reference

Returns repository reference metadata.

GET /api/releases

Returns list of past releases.

POST /api/releases

Creates a new release.

GET /api/releases/{version}

Returns release state.

POST /api/releases/{version}/stage2

Runs branch creation and merge process.

POST /api/releases/{version}/stage3

Creates merge requests.

9. GitLab APIs Used

Repository Compare:

GET /projects/:id/repository/compare

Create Branch:

POST /projects/:id/repository/branches

Merge Branches:

POST /projects/:id/repository/merges

Create Merge Request:

POST /projects/:id/merge_requests

Delete Branch:

DELETE /projects/:id/repository/branches/:branch

10. Frontend UI

Dashboard Page:

Shows list of releases and allows creation of a new release.

Release Detail Page:

Contains tabs:

- Stage 1 – Manage release candidates
- Stage 2 – Release branch status
- Stage 3 – Merge request status
- Cleanup – Removed candidate operations

Angular Material tables display repository status.

11. Error Handling

Merge conflicts do not stop execution for other repositories.

Failures are recorded in the release JSON file.

Users can retry failed operations via UI actions.

12. Security

GitLab Personal Access Token is stored only on the backend via environment variables.

Frontend never directly communicates with GitLab APIs.

13. Scalability Considerations

Target scale:

30 repositories per release cycle.

JSON storage is sufficient for this scale. If repository count grows significantly, migration to a database may be considered.

14. Future Enhancements

- CI/CD integration
- Automated conflict notifications
- Slack or Teams integration
- Release metrics dashboard
- Automated tagging and release notes generation